

【数大雁】

题目描述

一群大雁往南飞，给定一个字符串记录地面上的游客听到的大雁叫声，请给出叫声最少由几只大雁发出。

具体的：

- 1. 大雁发出的完整叫声为”quack“，因为有多只大雁同一时间嘎嘎作响，所以字符串中可能会混合多个”quack”。
- 2. 大雁会依次完整发出”quack”，即字符串中’q’, ’u’, ’a’, ’c’, ’k’ 这5个字母按顺序完整存在才能计数为一只大雁。如果不完整或者没有按顺序则不予计数。
- 3. 如果字符串不是由’q’, ’u’, ’a’, ’c’, ’k’ 字符组合而成，或者没有找到一只大雁，请返回-1。

输入描述

一个字符串，包含大雁quack的叫声。1 <= 字符串长度 <= 1000，字符串中的字符只有’q’, ’u’, ’a’, ’c’, ’k’。

输出描述

大雁的数量

用例

输入	quackquack
输出	1
说明	无
输入	qaauucqckk

输出	-1
说明	无

输入	quacqkuac
输出	1
说明	无

输入	qququaaauqccauqkkcauqqkcauuqkcaaukccakkck
输出	5
说明	无

题目解析

首先看一个例子：

q	u	a	c	q	k	q	u	a	c	k
---	---	---	---	---	---	---	---	---	---	---

CSDN @伏城之外

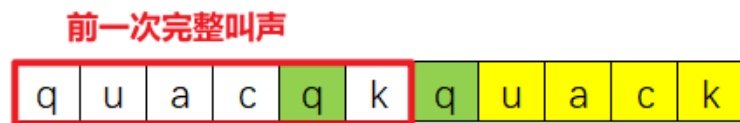
对于标黄的"uack"，我们应该归属于哪一个绿色的"q"呢？

- 如果归属于第一个绿色"q"，那么该例子就至少有两只大雁
- 如果归属于第二个绿色"q"，那么该例子就至少有一只大雁

这里的两个"q"更准确的描述是：

- 第一个"q"是前一次完整叫声中的第一个"q"

- 第二个"q"是前一次完整叫声后的第一个"q"



CSDN @伏城之外

下面我对这两种理解都做了实现

归属于第一个"q"解法 (90%通过率)

此解法虽然看起来和[LeetCode - 1419 数青蛙_伏城之外的博客-CSDN博客](#)

很像，但是难度却要大于 [leetcode](#) 这道题，原因是

大雁会依次完整发出"quack"，即字符串中'q','u','a','c','k' 这5个字母按顺序完整存在才能计数为一只大雁。如果不完整或者没有按顺序则不予计数。

而leetcode数青蛙

如果字符串 croakOfFrogs 不是由若干 有效的 "croak" 字符混合而成，请返回 -1 。

leetcode这题，如果存在不完整或者不按顺序的叫声，则直接返回-1。结束程序。

而本题，则对于不完整或者不按顺序的叫声，只是不予计数，后面还要继续统计。

比如 quaquack

如果按照leetcode数青蛙逻辑来做的话，结果应该返回-1。

而本题逻辑却需要返回1。

leetcode难度小的原因是，不需要考虑剔除不完整或者不按顺序的叫声。

而本题难度大的原因是，需要考虑剔除不完整或者不按顺序的叫声。

我的解题思路是，求出每个叫声quack的区间范围，即一次叫声的：[q索引位置，k索引位置]。

实现是：从左到右遍历叫声字符串，遇到q，则将其索引位置加入缓存中，

遇到u，则判断u出现的次数是否超过了q出现的次数，若是则忽略本次u，否则u次数++

遇到a，则判断a出现的次数是否超过了u出现的次数，若是则忽略本次a，否则a次数++

遇到c，则判断c出现的次数是否超过了a出现的次数，若是则忽略本次c，否则c次数++

遇到k，则判断c次数是否大于等于1，若否，则忽略本次k，否则出队第一个q出现索引和本次k的索引，组合成一个区间范围，然后u--,a--,c--，表示取出了一次叫声。

按照上面逻辑，我们可以剔除掉不完整或者不按顺序的叫声，并且获得每个合法叫声的区间范围。

接下来就是遍历每一个区间，和后面区间比较，看是否有交集，若有，则记录交集数。

最终最大的交集数就是最大雁数。

下图是用例3：qququaaauqccauqkkcauqqkcauuqkcaaukccakkck

各合法叫声区间的交集示意图

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35	36	37	38	39
q		u			a				c					k																									
	q			u		a				c					k																								
			q				u				a					c			k																				
								q				u					a				c					k													
													q				u				a						c			k									
																		u										a				c			k				
																			q						u				a				c			k			
																										q					u				a			c	k

CSDN @伏城之外

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35	36	37	38	39	
q		u			a				c					k																										
	q			u		a				c					k																									
		q					u				a					c			k																					
								q				u					a				c					k														
													q				u					a						c			k									
																		q						u					a				c			k				
																				q					u					a				c			k			
																										q					u				a			c	k	

CSDN @伏城之外

CSDN @伏城之外

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35	36	37	38	39		
q		u			a				c					k																											
	q			u		a				c					k																										
			q				u				a				c					k																					
								q				u				a					c						k														
													q				u					a					c			k											
																			q				u					a			c			k							
																					q					u				a			c			k					
																								q					u				a				c		k		

CSDN @伏城之外

CSDN @伏城之外

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35	36	37	38	39	
q		u			a				c					k																										
	q			u		a				c				k																										
			q				u			a				c						k																				
								q			u				a				c			k																		
												q				u					a							c				k								
																	q						a									c				k				
																								u					a				c				k			
																									q						u				a				c	k

CSDN @伏城之外

CSDN @伏城之外

JS算法源码

```

1  /* JavaScript Node ACM模式 控制台输入获取 */
2  const readline = require("readline");
3
4  const rl = readline.createInterface({
5    input: process.stdin,
6    output: process.stdout,
7  });

```

运行

```
8 | 9 | rl.on("line", (line) => {
10 |   console.log(getResult(line));
11 | });
12 |
13 | function getResult(quacks) {
14 |   let q, u, a, c, k;
15 |   q = [];
16 |   u = 0;
17 |   a = 0;
18 |   c = 0;
19 |
20 |   const range = [];
21 |
22 |   for (let i = 0; i < quacks.length; i++) {
23 |     switch (quacks[i]) {
24 |       case "q":
25 |         q.push(i);
26 |         break;
27 |       case "u":
28 |         if (u + 1 <= q.length) u++;
29 |         break;
30 |       case "a":
31 |         if (a + 1 <= u) a++;
32 |         break;
33 |       case "c":
34 |         if (c + 1 <= a) c++;
35 |         break;
36 |       case "k":
37 |         if (1 <= c) {
38 |           range.push([q.shift(), i]);
39 |           u--;
40 |           a--;
41 |           c--;
42 |         }
43 |         break;
44 |       default:
45 |         return -1;
```

```

46 |     }
    |     47 |     }
48 |
49 |     if (!range.length) return -1;
50 |
51 |     let max = 1;
52 |     for (let i = 0; i < range.length; i++) {
53 |         let count = 1;
54 |         for (let j = i + 1; j < range.length; j++) {
55 |             if (range[i][1] >= range[j][0]) count++;
56 |         }
57 |         max = Math.max(max, count);
58 |     }
59 |
60 |     return max;
61 | }

```

Java算法源码

```

1 | import java.util.ArrayList;
2 | import java.util.LinkedList;
3 | import java.util.Scanner;
4 |
5 | public class Main {
6 |     public static void main(String[] args) {
7 |         Scanner sc = new Scanner(System.in);
8 |         String quack = sc.next();
9 |         System.out.println(getResult(quack));
10 |    }
11 |
12 |    public static int getResult(String quack) {
13 |        LinkedList<Integer> q = new LinkedList<>();
14 |
15 |        int u = 0, a = 0, c = 0;

```



```

16
17 |     ArrayList<Integer[]> ranges = new ArrayList<>();
18
19     for (int i = 0; i < quack.length(); i++) {
20         switch (quack.charAt(i)) {
21             case 'q':
22                 q.add(i);
23                 break;
24             case 'u':
25                 if (u + 1 <= q.size()) u++;
26                 break;
27             case 'a':
28                 if (a + 1 <= u) a++;
29                 break;
30             case 'c':
31                 if (c + 1 <= a) c++;
32                 break;
33             case 'k':
34                 if (c >= 1) {
35                     ranges.add(new Integer[] {q.removeFirst(), i});
36                     u--;
37                     a--;
38                     c--;
39                 }
40                 break;
41             default:
42                 return -1;
43         }
44     }
45
46     if (ranges.size() == 0) return -1;
47
48     int ans = 1;
49     for (int i = 0; i < ranges.size(); i++) {
50         int count = 1;
51         for (int j = i + 1; j < ranges.size(); j++) {
52             if (ranges.get(i)[1] >= ranges.get(j)[0]) {

```

```

53         count++;54     }
55     }
56     ans = Math.max(ans, count);
57 }
58
59 return ans;
60 }
61 }

```

Python算法源码

```

1  # 输入获取
2  quacks = input()
3
4
5  # 算法入口
6  def getResult():
7      q = []
8      u, a, c = 0, 0, 0
9
10     rans = []
11
12     for i in range(len(quacks)):
13         char = quacks[i]
14
15         if char == 'q':
16             q.append(i)
17         elif char == 'u':
18             if u + 1 <= len(q):
19                 u += 1
20         elif char == 'a':
21             if a + 1 <= u:
22                 a += 1
23         elif char == 'c':

```

```

24         if c + 1 <= a:25 |             c += 1
25         elif char == 'k':
26             if c >= 1:
27                 rans.append([q.pop(0), i])
28                 u -= 1
29                 a -= 1
30                 c -= 1
31             else:
32                 return -1
33
34     if len(rans) == 0:
35         return -1
36
37     ans = 1
38     for i in range(len(rans)):
39         count = 1
40         for j in range(i + 1, len(rans)):
41             if rans[i][1] >= rans[j][0]:
42                 count += 1
43
44     ans = max(ans, count)
45
46     return ans
47
48 # 算法调用
49 print(getResult())

```

C算法源码

```

1  #include <stdio.h>
2  #include <math.h>
3  #include <string.h>
4
5  #define MAX_LEN 1001

```

```

6
7 | int shift(int q[], int *q_size) {
8     int res = q[0];
9
10    for (int i = 0; i < (*q_size) - 1; i++) {
11        q[i] = q[i + 1];
12    }
13
14    (*q_size)--;
15
16    return res;
17 }
18
19 int solution(char *s) {
20     int q[MAX_LEN] = {0};
21     int q_size = 0;
22
23     int u = 0;
24     int a = 0;
25     int c = 0;
26
27     int ranges[MAX_LEN][2];
28     int ranges_size = 0;
29
30     for (int i = 0; i < strlen(s); i++) {
31         if (s[i] == 'q') {
32             q[q_size++] = i;
33         } else if (s[i] == 'u') {
34             u = (int) fmin(u + 1, q_size);
35         } else if (s[i] == 'a') {
36             a = (int) fmin(a + 1, u);
37         } else if (s[i] == 'c') {
38             c = (int) fmin(c + 1, a);
39         } else if (s[i] == 'k') {
40             if (c >= 1) {
41                 ranges[ranges_size][0] = shift(q, &q_size);
42                 ranges[ranges_size][1] = i;

```

```

43         ranges_size++;
44         u--;
45         a--;
46         c--;
47     }
48     } else {
49         return -1;
50     }
51 }
52
53 if (ranges_size == 0) {
54     return -1;
55 }
56
57 int max = 1;
58 for (int i = 0; i < ranges_size; i++) {
59     int count = 1;
60     for (int j = i + 1; j < ranges_size; j++) {
61         if (ranges[i][1] >= ranges[j][0]) count++;
62     }
63     max = (int) fmax(max, count);
64 }
65
66 return max;
67 }
68
69 int main() {
70     char s[MAX_LEN];
71     scanf("%s", s);
72
73     printf("%d", solution(s));
74
75     return 0;
76 }

```

C++ 算法源码

```
1  #include <bits/stdc++.h>
2
3  using namespace std;
4
5  int solution(string &s) {
6      deque<int> q;
7
8      int u = 0;
9      int a = 0;
10     int c = 0;
11
12     vector<vector<int>> ranges;
13
14     for (int i = 0; i < s.length(); i++) {
15         switch (s[i]) {
16             case 'q':
17                 q.push_back(i);
18                 break;
19             case 'u':
20                 if (u + 1 <= q.size()) u++;
21                 break;
22             case 'a':
23                 if (a + 1 <= u) a++;
24                 break;
25             case 'c':
26                 if (c + 1 <= a) c++;
27                 break;
28             case 'k':
29                 if (c >= 1) {
30                     ranges.emplace_back(vector<int>{q.front(), i});
31                     q.pop_front();
32                     u--;
33                     a--;
34                     c--;
35                 }
36             }
```

```

36         break;
37         default:
38             return -1;
39     }
40 }
41
42 if (ranges.empty()) {
43     return -1;
44 }
45
46 int maxCount = 1;
47
48 for (int i = 0; i < ranges.size(); i++) {
49     int count = 1;
50
51     for (int j = i + 1; j < ranges.size(); j++) {
52         if (ranges[i][1] >= ranges[j][0]) count++;
53     }
54
55     maxCount = max(maxCount, count);
56 }
57
58 return maxCount;
59 }
60
61 int main() {
62     string s;
63     cin >> s;
64
65     cout << solution(s) << endl;
66
67     return 0;
68 }

```

归属于第二个"q"解法

此解法思路很简单，即假设每只大雁都连续不断的叫，即某只大雁叫完一次后，接着叫

我们以题目自带的用例4为例：

[illegible]

CSDN @伏城之外

其中一种颜色标记，就是一只大雁的多次叫声，比如黄色标记，就相当于一只大雁叫了三声quack，这三声quack是不交叉的，因此可以认为是同一只大雁叫的。

这里其实有贪心思维的存在，即认为每个大雁都会发出多次叫声，只要存在多次不交叉的叫声，即认为是一只大雁发出的，这样最终就可以得到最少大雁数量。

具体实现时，需要对输入的字符串进行多轮遍历，每轮遍历确认一只大雁，并将该大雁的多次叫声对应的位置标记为used，这样到下一轮遍历时，就不会造成多个大雁使用同一个叫声。

Java算法源码

```

1  import java.util.Scanner;
2
3  public class Main {
4      public static void main(String[] args) {
5          Scanner sc = new Scanner(System.in);
6          System.out.println(getResult(sc.next()));
7      }
8
9      public static int getResult(String s) {
10         char[] quacks = s.toCharArray();
11
12         int count = 0;
13         while (find(quacks)) {
14             count++;
15         }
16
17         // 如果没有找到一只大雁，请返回-1。

```



```

18     return count == 0 ? -1 : count;
19     }
20
21 public static boolean find(char[] quacks) {
22     boolean isFind = false;
23
24     int index = 0;
25     int[] quack_index = new int[5];
26
27     for (int i = 0; i < quacks.length; i++) {
28         if (quacks[i] == "quack".charAt(index)) {
29             quack_index[index++] = i;
30         }
31
32         if (index == 5) {
33             isFind = true;
34
35             for (int j : quack_index) {
36                 quacks[j] = ' ';
37             }
38
39             index = 0;
40         }
41     }
42
43     return isFind;
44 }
45 }

```

JS算法源码

```

1  const rl = require("readline").createInterface({ input: process.stdin });
2  var iter = rl[Symbol.asyncIterator]();
3  const readline = async () => (await iter.next()).value;
4
5  void (async function () {

```

```
6 | console.log(getResult(await readline())); 7 |})();
8 |
9 | function getResult(s) {
10 |     const quacks = [...s];
11 |
12 |     let count = 0;
13 |     // 多轮找大雁
14 |     while (find(quacks)) {
15 |         count++;
16 |     }
17 |
18 |     return count == 0 ? -1 : count;
19 | }
20 |
21 | function find(quacks) {
22 |     let isFind = false;
23 |
24 |     let index = 0;
25 |     const quack_index = new Array(5).fill(-1);
26 |
27 |     for (let i = 0; i < quacks.length; i++) {
28 |         if (quacks[i] == "quack"[index]) {
29 |             quack_index[index++] = i;
30 |         }
31 |
32 |         if (index == 5) {
33 |             isFind = true;
34 |
35 |             for (let j of quack_index) {
36 |                 quacks[j] = "";
37 |             }
38 |
39 |             index = 0;
40 |         }
41 |     }
42 |
43 |     return isFind;
```

Python算法源码

```
1 # 输入获取
2 quacks = list(input())
3
4
5 # 一轮找出一只大雁的所有叫声
6 def find():
7     isFind = False
8
9     index = 0
10    quack_index = [-1, -1, -1, -1, -1]
11
12    for i in range(len(quacks)):
13        if quacks[i] == "quack"[index]:
14            quack_index[index] = i
15            index += 1
16
17        if index == 5:
18            isFind = True
19
20            for j in quack_index:
21                quacks[j] = ' '
22
23            index = 0
24
25    return isFind
26
27
28 # 算法入口
29 def getResult():
30    count = 0
31
32    while find():
```

```
33 |         count += 1
34 |
35 |     return -1 if count == 0 else count
36 |
37 |
38 | # 算法调用
39 | print(getResult())
```

C算法源码

```
1  #include <stdio.h>
2  #include <string.h>
3  #include <stdbool.h>
4  #include <stdlib.h>
5
6  #define MAX_LEN 1001
7
8  bool find(char* s) {
9      bool isFind = false;
10
11      int index = 0;
12      int quack_index[5] = {0};
13
14      for (int i = 0; i < strlen(s); i++) {
15          if (s[i] == "quack"[index]) {
16              quack_index[index++] = i;
17          }
18
19          if (index == 5) {
20              isFind = true;
21
22              for (int j = 0; j < 5; j++) {
23                  s[quack_index[j]] = ' ';
24              }
25      }
```

```

26         index = 0;
27     }
28 }
29
30 return isFind;
31 }
32
33 int main() {
34     char s[MAX_LEN];
35     scanf("%s", s);
36
37     int count = 0;
38     while (find(s)) {
39         count++;
40     }
41
42     printf("%d", count == 0 ? -1 : count);
43
44     return 0;
45 }

```

C++ 算法源码

```

1  #include <bits/stdc++.h>
2
3  using namespace std;
4
5  bool find(string &s, vector<bool> &used) {
6      bool isFind = false;
7
8      int index = 0;
9      vector<int> quack_index(5, 0);
10
11     for (int i = 0; i < s.length(); i++) {
12         if (!used[i] && s[i] == "quack"[index]) {

```

```

13         quack_index[index++] = i;14     }
15
16     if (index == 5) {
17         isFind = true;
18
19         for (int j = 0; j < 5; j++) {
20             used[quack_index[j]] = true;
21         }
22
23         index = 0;
24     }
25 }
26
27 return isFind;
28 }
29
30 int main() {
31     string s;
32     cin >> s;
33
34     vector<bool> used(s.length(), false);
35
36     int count = 0;
37     while (find(s, used)) {
38         count++;
39     }
40
41     cout << (count == 0 ? -1 : count) << endl;
42
43     return 0;
44 }

```